

Práctico AYED1 - 14-09

Práctico 3 - Ejercicio 1b

$\text{sum_ant} : [\text{Int}] \rightarrow \text{Bool}$
 $\text{sum_ant.xs} = \langle \exists i : 0 \leq i < \#xs : xs ! i = \text{sum}.(xs \uparrow i) \rangle$

¿Qué calcula esta función? Me dice si hay algún elemento de xs que es igual a la suma de los anteriores.

Chat: un ejemplo en el que vale sum_ant.xs , y otro en el que vale $\neg \text{sum_ant.xs}$

Derivación: La haremos por inducción en xs.

Caso base: $xs = []$
 $\text{sum_ant}.[]$
 $= \{ \text{ejercicio} \}$
 False

Paso inductivo: $xs = y \blacktriangleright ys$

H.I. : $\text{sum_ant.ys} = \langle \exists i : 0 \leq i < \#ys : ys ! i = \text{sum}.(ys \uparrow i) \rangle$

$\text{sum_ant}.(y \blacktriangleright ys)$
 $= \{ \text{esp.} \}$
 $\langle \exists i : 0 \leq i < \#(y \blacktriangleright ys) : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle$

= { def. # y lógica }
 $\langle \exists i : i = 0 \vee 1 \leq i < \#ys + 1 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle$
 = { part. rango }
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle \vee$
 $\langle \exists i : 1 \leq i < \#ys + 1 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle$
 = { buscamos la H.I. en la 2da cuantificación: cambio de variable $i \rightarrow i + 1$ }
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle \vee$
 $\langle \exists i : 1 \leq i + 1 < \#ys + 1 : \underline{(y \blacktriangleright ys) ! (i+1)} = \text{sum}(\underline{(y \blacktriangleright ys) \uparrow (i+1)}) \rangle$
 = { def ! y def $\uparrow$ }
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle \vee$
 $\langle \exists i : \underline{1 \leq i + 1 < \#ys + 1} : \underline{ys ! i = \text{sum}(y \blacktriangleright (ys \uparrow i))} \rangle$
 = { resto 1 en la desigualdad del rango, def. sum }
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = \text{sum}((y \blacktriangleright ys) \uparrow i) \rangle \vee$
 $\langle \exists i : 0 \leq i < \#ys : ys ! i = y + \text{sum}(ys \uparrow i) \rangle$

Acá frenamos. La H.I. era: $\text{sum_ant.ys} = \langle \exists i : 0 \leq i < \#ys : ys ! i = \text{sum}(ys \uparrow i) \rangle$
 PERO NO PODEMOS LLEGAR A ESO PORQUE ESTA LA “y +” MOLESTANDO.
 Además no hay ninguna propiedad ni teroema ni axioma matemático ni lógico
 que me permitan sacar esa “y +”. **Debo generalizar.**

$\text{sum_ant_gen} : [\text{Int}] \rightarrow \text{Int} \rightarrow \text{Bool}$
 $\text{sum_ant_gen.xs.k} = \langle \exists i : 0 \leq i < \#xs : xs ! i = k + \text{sum}(xs \uparrow i) \rangle$

Generalizamos así por dos razones:

1. Si resuelvo este problema (el de sum_ant_gen), resuelvo el otro (sum_ant).
 Ya que sum_ant_gen **generaliza** a sum_ant , o equivalentemente,

sum_ant es un caso particular de sum_ant_gen.

Demostración:

```
sum_ant_gen.xs.0
= { esp. sum_ant_gen }
  <  $\exists i : 0 \leq i < \#xs : xs ! i = 0 + \text{sum.}(xs \uparrow i)$  >
= { neutro + }
  <  $\exists i : 0 \leq i < \#xs : xs ! i = \text{sum.}(xs \uparrow i)$  >
= { esp. sum_ant }
  sum_ant.xs
```

Luego, podemos definir sum_ant en una sola línea (o sea, **no por inducción**):

$\text{sum_ant.xs} \doteq \text{sum_ant_gen.xs.0}$

2. La otra razón es que cuando derive sum_ant_gen por inducción, ya voy viendo que vo+ y a poder aplicar H.I. Lo confirmaremos al derivar.

Derivación de sum_ant_gen: Por inducción en xs.

Caso base: $xs = []$
sum_ant_gen.[].k
= { esp. }
< $\exists i : 0 \leq i < \#[] : [] ! i = k + \text{sum.}([] \uparrow i)$ >
= { def # y rango vacío }
False

Paso inductivo: $xs = y \blacktriangleright ys$

H.I.: $\text{sum_ant_gen.ys.j} = \langle \exists i : 0 \leq i < \#ys : ys ! i = j + \text{sum.}(ys \uparrow i) \rangle$ **para cualquier j**

$\text{sum_ant_gen.}(y \blacktriangleright ys).k$
 $= \{ \text{esp.} \}$
 $\langle \exists i : 0 \leq i < \#(y \blacktriangleright ys) : (y \blacktriangleright ys) ! i = k + \text{sum.}((y \blacktriangleright ys) \uparrow i) \rangle$
 $= \{ \text{mismos pasos que la derivación de sum_ant pero cuidando que esté el "k +" en todos lados} \}$
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = k + \text{sum.}((y \blacktriangleright ys) \uparrow i) \rangle \quad \vee$
 $\langle \exists i : 0 \leq i < \#ys : ys ! i = k + (y + \text{sum.}(ys \uparrow i)) \rangle$
 $= \{ \text{asociatividad de la +} \}$
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = k + \text{sum.}((y \blacktriangleright ys) \uparrow i) \rangle \quad \vee$
 $\langle \exists i : 0 \leq i < \#ys : ys ! i = (k + y) + \text{sum.}(ys \uparrow i) \rangle$
 $= \{ \text{H.I.} \}$
 $\langle \exists i : i = 0 : (y \blacktriangleright ys) ! i = k + \text{sum.}((y \blacktriangleright ys) \uparrow i) \rangle \quad \vee \quad \text{sum_ant_gen.ys.}(k + y)$
 $= \{ \text{rango unitario} \}$
 $(y \blacktriangleright ys) ! 0 = k + \text{sum.}((y \blacktriangleright ys) \uparrow 0) \quad \vee \quad \text{sum_ant_gen.ys.}(k + y)$
 $= \{ \text{def ! y def } \uparrow \}$
 $y = k + \text{sum.}[] \quad \vee \quad \text{sum_ant_gen.ys.}(k + y)$
 $= \{ \text{def sum} \}$
 $y = k \quad \vee \quad \text{sum_ant_gen.ys.}(k + y)$

Resultado final:

$\text{sum_ant.xs} \doteq \text{sum_ant_gen.xs.0}$

$\text{sum_ant_gen.[]} . k \doteq \text{False}$

$\text{sum_ant_gen.}(x \blacktriangleright xs).k \doteq x = k \quad \vee \quad \text{sum_ant_gen.xs.}(k + x)$

Veamos la función en acción:

sum_ant.[1,2,3] → sum_ant_gen.[1,2,3].0
→ 1 = 0 ∨ sum_ant_gen.[2, 3].(0 + 1)
→ 1 = 0 ∨ sum_ant_gen.[2, 3].1
→ 1 = 0 ∨ (2 = 1 ∨ sum_ant_gen.[3].(1 + 2))
→ 1 = 0 ∨ (2 = 1 ∨ (3 = 3 ∨ sum_ant_gen.[].(3 + 3)))
→ 1 = 0 ∨ (2 = 1 ∨ (3 = 3 ∨ False))
→ False ∨ (False ∨ (True ∨ False))
..... (salteo pasos)
→ True.

RECREO. VOLVEMOS 12:10

Práctico 3 - Ejercicio 2a

Especificar y derivar:

esCuad : Nat → Bool, dado un natural determina si es el cuadrado de un número.

Especificación:

$\text{esCuad}.n = \langle \exists i :: n = i * i \rangle$

Derivación:

Antes de hacer inducción, transformamos un poco la especificación:

$\text{esCuad}.n$
 $= \{ \text{esp.} \}$
 $\langle \exists i : n = i * i \rangle$
 $= \{ \text{sabemos que } n \geq 0, \text{ luego } i \geq 0 \}$
 $\langle \exists i : 0 \leq i : n = i * i \rangle$
 $= \{ \text{más algebra: también sabemos que } i \leq n \}$
 $\langle \exists i : 0 \leq i \leq n : n = i * i \rangle$

Esto nos gusta más para derivar, porque el i tiene una cantidad finita de valores posibles.
 Ahora sí, hagamos inducción en n .

Caso base: $n = 0$

$\text{esCuad}.0$
 $= \{ \text{esp.} \}$
 $\langle \exists i : 0 \leq i \leq 0 : 0 = i * i \rangle$
 $= \{ \text{lógica} \}$
 $\langle \exists i : i = 0 : 0 = i * i \rangle$
 $= \{ \text{rango unitario} \}$
 $0 = 0 * 0$
 $= \{ \text{lógica} \}$
 True

Paso inductivo: $n = m + 1$.

H.I.: $\text{esCuad}.m = \langle \exists i : 0 \leq i \leq m : m = i * i \rangle$

$\text{esCuad}.(m + 1)$
 $= \{ \text{esp.} \}$

$$\langle \exists i : 0 \leq i \leq m+1 : m+1 = i * i \rangle$$

= { l3gica }

$$\langle \exists i : 0 \leq i \leq m \ \forall i = m+1 : m+1 = i * i \rangle$$

= { part. de rango }

$$\langle \exists i : 0 \leq i \leq m : m+1 = i * i \rangle \vee$$

$$\langle \exists i : i = m+1 : m+1 = i * i \rangle$$

Ac3 nos trabajamos. No hay forma de llegar a la H.I. porque molesta el "+ 1".
Debemos generalizar:

esCuad_gen : Nat → Nat → Bool
 esCuad_gen.n.k = $\langle \exists i : 0 \leq i \leq n : n + k = i * i \rangle$

Nos sirve esto por dos razones que veremos:

1. esCuad_gen generaliza a esCuad
2. vamos a poder derivar por inducci3n esCuad_gen sin problemas.

1. esCuad_gen generaliza a esCuad:

esCuad_gen.n.0
 = { esp esCuad_gen }
 $\langle \exists i : 0 \leq i \leq n : n + 0 = i * i \rangle$
 = { arit }
 $\langle \exists i : 0 \leq i \leq n : n = i * i \rangle$
 = { esp esCuad }
 esCuad.n

Luego podemos definir:

$\text{esCuad}.n \doteq \text{esCuad_gen}.n.0$

2. Derivación de **esCuad_gen**:

Por inducción en n .

Caso base: $n = 0$.

$\text{esCuad_gen}.0.k$
 $= \{ \text{esp.} \}$
 $\langle \exists i : 0 \leq i \leq 0 : 0 + k = i * i \rangle$
 $= \{ \text{rango unitario: } i = 0 \}$
 $0 + k = 0 * 0$
 $= \{ \text{arit} \}$
 $k = 0$

Listo! Llegué a algo que es parte del lenguaje de programación.

No hace falta y queda muuy feo hacer el análisis por casos:

$(k = 0 \rightarrow \text{True}$
 $\square k \neq 0 \rightarrow \text{False}$
 $)$

Paso inductivo: $n = m + 1$.

H.l.: $\text{esCuad_gen}.m.j = \langle \exists i : 0 \leq i \leq m : m + j = i * i \rangle$ **para todo j**

$\text{esCuad_gen}.(m+1).k$
 $= \{ \text{esp.} \}$
 $\langle \exists i : 0 \leq i \leq m + 1 : (m + 1) + k = i * i \rangle$

$= \{ \text{lógica y part. rango} \}$
 $\langle \exists i : 0 \leq i \leq m : (m+1) + k = i * i \rangle \vee$
 $\langle \exists i : i = m+1 : (m+1) + k = i * i \rangle$
 $= \{ \text{asociatividad} + \}$
 $\langle \exists i : 0 \leq i \leq m : m + (1+k) = i * i \rangle \vee$
 $\langle \exists i : i = m+1 : (m+1) + k = i * i \rangle$
 $= \{ \text{H.I.} \}$
 $\text{esCuad_gen.m.(1+k)} \vee \langle \exists i : i = m+1 : (m+1) + k = i * i \rangle$
 $= \{ \text{rango unitario} \}$
 $\text{esCuad_gen.m.(1+k)} \vee (m+1) + k = (m+1) * (m+1)$

Listo! **Resultado final:**

$\text{esCuad.n} \doteq \text{esCuad_gen.n.0}$

$\text{esCuad_gen.0.k} \doteq k = 0$

$\text{esCuad_gen.(n+1).k} \doteq (n+1) + k = (n+1) * (n+1) \vee \text{esCuad_gen.n.(1+k)}$

Veamos la función en acción:

$\text{esCuad.4} \rightarrow \text{esCuad_gen.4.0}$
 $\rightarrow 4 + 0 = 4 * 4 \vee \text{esCuad_gen.3.(1+0)}$
 $\rightarrow 4 + 0 = 4 * 4 \vee (3 + 1 = 3 * 3 \vee \text{esCuad_gen.2.(1+1)})$
 $\rightarrow 4 + 0 = 4 * 4 \vee$
 $3 + 1 = 3 * 3 \vee (2 + 2 = 2 * 2 \vee \text{esCuad_gen.1.(1+2)})$
 $\rightarrow 4 + 0 = 4 * 4 \vee$
 $3 + 1 = 3 * 3 \vee$

$2 + 2 = 2 * 2 \vee (1 + 3 = 1 * 1 \vee \text{esCuad_gen.0.(1+3)})$

→ $4 + 0 = 4 * 4 \vee$

$3 + 1 = 3 * 3 \vee$

$2 + 2 = 2 * 2 \vee$

$1 + 3 = 1 * 1 \vee (4 = 0)$

→ $4 = 4 * 4 \vee$

$4 = 3 * 3 \vee$

$4 = 2 * 2 \vee$ ---> acá está

$4 = 1 * 1 \vee$

$4 = 0 * 0$

→ $4 = 4 * 4 \vee$

$4 = 3 * 3 \vee$

True $\vee$

$4 = 1 * 1 \vee$

$4 = 0 * 0$

..... (salteo)

→ True